

Distributed computing, tomorrow's panacea — an introduction to current technology

S J T Condie

Distributed computing, driven by industry 'hype', has raised expectations of highly functional systems, delivered at reasonable cost, solving the majority of computing problems.

This paper will investigate how closely distributed computing technologies, conveniently termed 'middleware', have developed to meet these expectations. The status of current standards, such as CORBA and DCE, will be reviewed. The review will range from a discussion of the standards in terms of functionality, IIOP for example, to the de facto standard offerings from Microsoft such as DCOM. The expected evolution path in terms of convergence or divergence of standards and technologies will be examined.

The role of current technologies as they are being applied to the telecommunications industry will be investigated, presenting a view finally on how these technologies, sufficiently matured, may be applied to the telecommunications networks of the future.

1. Introduction

Why would anyone want a distributed computer system? There are many computer applications, addressing particular requirements, that were designed and work perfectly well as monolithic systems and there is no reason to believe that this situation will change. However, distributed systems have certain advantages over monolithic systems.

Resources within the distributed system such as printers, file servers and other computers, and the data present within those computers and file systems, may be shared across all the computers within the system. This aspect of working together enables great flexibility, robustness and potentially limitless scalability.

In the event of a failure, both hardware and software needs to continue to function to provide a service. Distributed systems can be developed with fault tolerance in mind to enable non-stop availability.

By making use of idle processing power within the distributed system the efficiency of the system can be increased by 'load balancing' across all the processors. Above all, distributed systems offer the prospect of better price/performance over their monolithic predecessors.

However, the use of distributed systems presents a number of challenges not present in monolithic ones. It is

generally agreed that distributed systems are harder to design, harder to implement, and harder to operate and effectively manage than monolithic systems. Distributed systems contain many more components than monolithic systems and consequently can have multiple failure modes. Distributed systems, by their nature, can also have very complex processing flows, and inherently there will also be complex state within the system. Synchronisation across a distributed system presents a management challenge that requires specialised technologies to meet, and security enforcement can also be less than trivial to achieve. Given that distributed systems contain so many diverse technologies, which may or may not work together well, and such complex processing models, it is also very difficult to estimate the performance of an entire system.

However, regardless of the challenges, distributed, or network-based, computing has become increasingly important to businesses, offering new flexibility, new ways of integrating existing and new IT assets, and new ways of facilitating transactions between different parts of the business. Additionally, it has the potential to provide ways of enabling business transactions between the business and its partners, and between a business and its customers, and all with the benefits of reduced cost of ownership.

This panacea, driven by industry 'hype', has raised expectations of highly functional systems solving the

majority of computing problems. The reality is that early distributed technology offerings have been unable to adequately satisfy these expectations except in the simplest of scenarios.

This paper will introduce some of the concepts of distributed computing technologies, conveniently termed 'middleware', examine some of the advantages and disadvantages in its use, and subsequently discuss some of the current 'middleware' standards in terms of functionality. The paper will conclude with a look at how current distributed computing technologies are being applied to the telecommunications industry and give a view on how these technologies, when sufficiently matured, may provide solutions for the telecommunications networks and products of the future.

2. Middleware

Distributed operating systems extend the concept of a virtual machine over a number of interconnected computers or hosts. Open distributed systems (the subject of this paper) [1] differ from distributed operating systems in a fundamental way, namely that they provide support for distributed applications in each host in an open manner. Distributed systems can be defined as combinations of hardware, software and network components in which software components execute on two or more processors, communicating via a network, providing the necessary application layer support.

This is achieved by a software layer, which can be termed 'middleware', between the communications facilities provided on each host and the application. This layer ensures a high degree of independence between the underlying systems and the applications. Each host's operating system must be provided with a software 'tool-set' to adequately support the set of distributed applications required for the host to participate in the distributed system.

Middleware can be defined as software that is used to move information from one program to one or more other programs, shielding the application developer from dependencies inherent in communications protocols, operating systems, and hardware platforms. More formally, middleware provides a mediation service between application programs and networks, managing the interactions between disparate applications across heterogeneous computing platforms. Generally, middleware provides a set of services specifically geared towards supporting distributed computing. These services ensure that any distributed application is scalable, reliable, secure, available for use and gives high performance — in short, middleware services provide the necessary 'plumbing' (Fig 1).

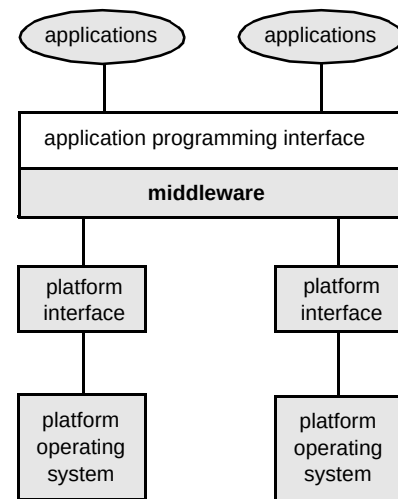


Fig 1 Middleware.

The objectives of middleware can be achieved by the use of a number of different approaches. Some vendors have chosen to implement their particular middleware solutions to use one or more particular approaches. Some of the approaches are described below and more or less align with the middleware market segments (see the Ovum Web site as a source of objective material [2]).

2.1 Database middleware

Database middleware is probably the most mature of the different middleware approaches and can be viewed as a 'data centric' architecture. Database middleware generally implies the use of database gateways. Client programs issue structured query language (SQL) requests to the gateways, which in turn pass the requests on to the required database. If required, the gateway will translate the request to the native 'dialect' of the target database. In enabling communications between applications, the database acts as a central repository for information. Database middleware implies synchronous, point-to-point communication. Unfortunately, database middleware has a number of limitations. The approach does not work well in applications having high performance requirements, as the database rapidly becomes a bottle-neck. Also, the system will not work when faced with network failures — when access to the database is lost, no communication can occur.

2.2 Distributed transaction processing monitors (DTPMs)

DTPMs provide tools and an environment for developing and deploying distributed applications, but are typically not used for general-purpose application-to-application communication. They provide the environment for transactional applications, usually involving relational databases, and adhering to the ACID properties (atomicity, consistency, isolation, and durability). TP monitors provide application development tools (such as user interaction and

database interfaces), system administration (such as security and tuning), and transaction execution (such as scheduling and load balancing). In TP monitors, clients invoke remote procedures that reside on servers, which may also contain an SQL database engine. Procedural statements on the server execute a group of SQL statements or transactions, which either all succeed or all fail as a transactional unit. The transactional applications are called 'on-line transaction processing' (OLTP) and tend to be mission critical, requiring a rapid response 100% of the time and tight controls over the security and integrity of the database.

TP monitors are generally best employed where there are requirements for transactions that need to be co-ordinated and synchronised across multiple databases. TP monitors tend to be heavyweight and expensive, requiring considerable expertise to implement properly.

2.3 Remote procedure calls

Remote procedure calls (RPCs) have been around for some time and from a programmer's perspective are easy to understand. The call invokes a remotely located procedure and the results are returned. RPCs enable the logic of an application to be distributed across the network. Program logic on remote systems can be executed as simply as calling a local routine. When using RPCs, little communication code needs to be written, most of the code being generated from the interface definition language (IDL). Generally, the application components communicate with each other synchronously in a 'request, wait for reply' manner. RPCs work well for smaller, simpler applications where all communication is one to one and asynchronous operation is not a requirement. They do not scale well to larger, mission-critical applications, because many crucial details are left to the programmer to implement. Such details are:

- handling network or system failures,
- handling multiple connections,
- portability,
- buffering and flow control,
- synchronisation between processes.

Due to their synchronous nature, RPCs are not a good choice for enterprise-spanning applications where both high performance and high reliability are required.

2.4 Distributed Computing Environment (DCE)

The DCE is a standard originally developed by the Open Software Foundation (OSF) but now owned by the Open Group who provide source code for vendors to provide licensed implementations. The DCE makes extensive use of RPC mechanisms, but is much more functionally rich. The DCE comprises a set of services with a communications

layer based on RPCs, which are used to provide program-to-program communication. Additionally, RPCs are used to invoke the services of the DCE and, internally, all the run-time services of the DCE use RPCs for communication. Run-time services include security, thread services, time synchronisation, directory service and a distributed file service. The DCE run-time environment consists of hosts that run DCE client software or DCE server software or both. There are a number of add-on products that provide additional functionality missing in the base product. An example is distributed transaction processing. The DCE, although providing an excellent technical architecture for distributed computing, is expensive and requires a high degree of technical competence to implement even simple applications.

2.5 Object Request Brokers (ORBs)

ORBs can be considered as the language-independent mechanisms, which enable the objects that comprise an application to be distributed, shared and invoked across heterogeneous networks. Currently the most important ORBs are CORBA-compliant ORBs, DCOM (Microsoft's Distributed Component Object Model) and Java RMI (Remote Method Invocation), although RMI does not have the language independence of CORBA and DCOM. There are also other proprietary ORBs available. ORBs are designed to be used in applications using a strict object-oriented approach, where 'objects' are the 'only way'. An IDL is used to define the interfaces between objects (in a manner similar to RPCs where IDL is used to define interfaces to procedures). ORBs work best in circumstances where the interfaces between objects do not change frequently, requiring code to be recompiled and relinked. CORBA specifies a dynamic invocation interface to address this problem, but this can prove difficult to use. ORBs, like RPCs are generally synchronous, operating in a point-to-point fashion. Generally, both CORBA and DCOM assume a reliable communications layer and do not provide services to deal with failures. Early ORB implementations did not perform or scale particularly well, precluding their extensive use in real-time or near real-time applications.

2.6 Message-oriented middleware (MOM)

MOM can be viewed as 'process centric' as it provides process-to-process data exchange, enabling the creation of distributed applications. MOM is analogous to e-mail in the sense that it is asynchronous, requiring the recipients of messages to interpret their meaning and to take appropriate action. Messages usually consist of control information and application data, the control data being used by the messaging software. Unlike ORBs and RPC mechanisms, MOM does not assume reliable transport layers. Consequently products provide support for integrity, guaranteed delivery, data format translation and protection of message queues. Two message-oriented middleware

paradigms have emerged, namely message passing and message queuing.

3. Standards

3.1 DCE

The DCE infrastructure was designed to support the implementation and integration of client/server applications while hiding the inherent complexity of the distributed processing from the user. DCE is intended to form a comprehensive software platform on which distributed applications can be built, executed, and maintained [3].

The Distributed Computing Environment (DCE), originally developed by the Open Systems Foundation (OSF), provides an integrated distributed environment incorporating technology from diverse vendors, but having a common original code base. DCE is now controlled by the Open Group which provides a reference implementation (source code) that is independent of network and operating system, on which all DCE products are based, with the intention that the licensed implementations will include mature, proven technology that can be used in parts, as individual services, or as a complete integrated infrastructure. DCE provides an architecture in which new technologies can be included, thus allowing for future enhancements.

DCE is comprised of a set of integrated system services providing an interoperable and flexible distributed environment having the primary goal of solving interoperability problems in heterogeneous, networked environments. DCE is available on different hardware and software platforms ranging from the many flavours of Unix, WindowsNT [4], DEC's VMS, and Macintosh, through to IBM mainframes. There are also many products that utilise the underlying services provided by DCE to support the additional functionality provided by their product. DCE has a communications layer based on RPC technology, which indeed provides the foundation of DCE. RPCs are used to provide program-to-program communication and are used to invoke the services of the DCE. Internally, all the run-time services of the DCE use RPCs for communication.

The run-time services include a set of fundamental distributed services providing tools for software developers to create the end-user services needed for distributed computing. They include:

- as previously mentioned, an RPC mechanism, facilitating client/server communication and providing portability, network independence, and secure distributed applications,
- a directory service, providing X.500 support and a single naming model to allow application developers to identify and access distributed resources,
- a distributed time service, providing a mechanism to monitor and track clocks in a distributed environment, together with accurate time-stamps to reduce the load on the system administrator,
- a distributed security service, providing the network with authentication, authorisation, and user account management services to maintain the integrity, privacy, and authenticity of the distributed system,
- a thread service, which provides a simple, portable, programming model for building concurrent applications,
- a distributed file system, interoperating with the native network file system to provide a high-performance, scalable and secure file-access system,
- support for diskless workstations to use disks on servers, thereby reducing the need of local disks, and enabling performance enhancements to reduce network overhead.

Figure 2 depicts a simplified view of the architecture and services of the DCE.

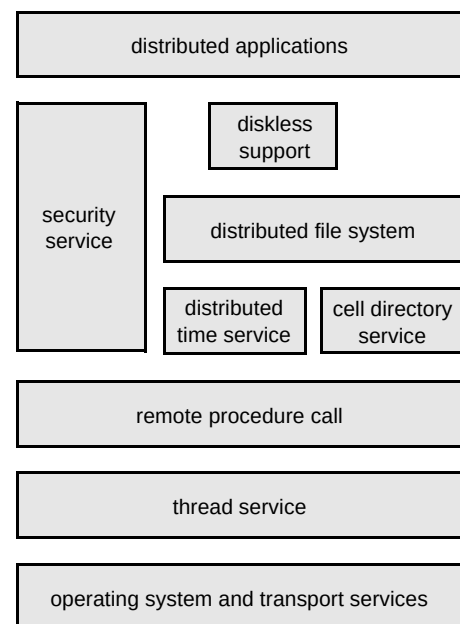
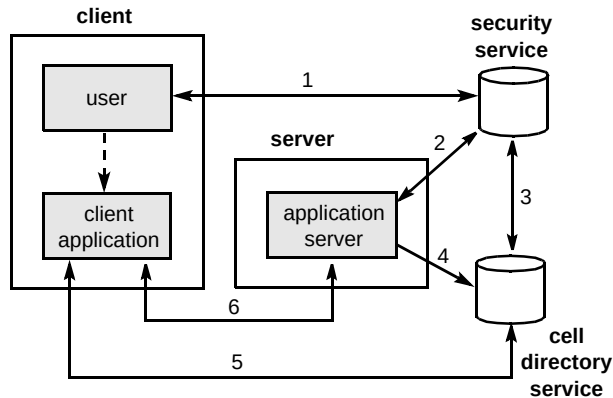


Fig 2 The architecture and services of the DCE.

In the DCE the basic unit of operation and administration is the cell, which is a group of users, systems and resources that need to share common DCE services. Each resource in a DCE cell, which may be a user, file, print server, application server, DCE service, device, node or other resource on the network, has a global name, which is universally meaningful and usable from anywhere in the

DCE naming environment, and a local or cell name, usable from within the home cell.

A key component of the DCE is the directory service. By using the directory service, any resource can be contacted anywhere on the network by using its unique global name. The directory service, on being given the name, returns the network address of that resource. Figure 3 illustrates how a user invokes a DCE application.



1 The user logs on to the DCE, supplying a password to the security service to verify their identity. The user can then invoke a client application. 2 When a server application or DCE service starts it also logs on to the DCE through a programming call to verify its identity. 3 The cell directory service verifies the server's identity. 4 The server places its binding information into the cell directory service. 5 The client obtains the server's binding information from the cell directory service. 6 The client invokes the server application via an RPC.

Fig 3 How the DCE works.

DCE application development

Most DCE applications will use RPCs and the key to the process is the definition of the interface. The interface defines the name of each procedure, the data types for each argument, their order, the return value and its data type, and whether the arguments are input, output or both. Individual interfaces are distinguished from others by a universal unique identifier (UUID). A utility called UUIDGEN can be used to produce a UUID plus the template for the interface definition.

The programmer defines the interface using the IDL, which in DCE looks very like the declaration syntax and semantics of the C programming language. Programmers use a text editor to write or modify IDL declarations:

```

[
uuid(005E9AC9 — 486D — 186C — 0000C07C3610),
version(1.0)
]
interface myInterface
{
    some more declarations....
}
  
```

The interface definition is then stored in the interface definition file (IDF). When the IDF is complete, the programmer uses the IDL compiler to generate client and server stub code and application header files in C. Originally, DCE could only be used to generate C stubs, but recent enhancements have resulted in support for other languages, such as C++, to include features such as inheritance and object references in support of object-oriented applications. This feature supports adoption of any object model or class hierarchy, providing developers with additional flexibility.

Programmers must add the coding needed to handle errors within application code, and DCE routines are made available to help with this. Developers must also add the code to use other DCE services, such as the threads service, the security service and the directory service.

Although DCE provides an excellent technical architecture to support distributed applications, it is expensive and complex and requires a high level of resource commitment. Many companies using DCE employ permanent teams of experienced technical people, whose only role is to provide simplified interfaces and add-on services to the basic core services. DCE is middleware that can be used as infrastructural software for the company, but it does not provide all of the essential services for it to be used to support Internet-based applications.

3.2 CORBA

The Object Management Group (OMG) [5] is a consortium that was formed in 1989 with the remit of developing an object-oriented environment to integrate different IT platforms. Deriving from the requirements of the Object Management Architecture (OMA), a set of standards for a Common Object Request Broker Architecture (CORBA) were the result, with version 1.0, specifying a basic ORB, being released in 1991. The CORBA 2.0 architecture [6] followed in 1994, permitting different ORBs to work together.

The CORBA standard consists of a specification for an ORB together with specifications for a number of services that add value to the basic ORB. These additional services are known as CORBAServices. The location of these services is not mandated by the standard, permitting them to be on the server, on a separate host, on the client, or on both.

The CORBA standard focuses on the use of distributed objects to provide for distributed systems integration. The benefit in adopting this approach is encapsulation — data and state are only accessible through invocations of a set of defined operations rather than through direct access. The use of objects makes dealing with different implementations of the same service much easier, because differences in data

representation are hidden. This makes application and system integration easier.

The CORBA infrastructure (Fig 4) consists of the following components:

- the object request broker, which is the object bus, providing the communication infrastructure through which applications access services and facilities, and through which objects interact with each other, allowing objects to transparently communicate with other objects, both locally and remote,
- the set of common object services, which are objects providing the ORB with services that are useful across most applications, a sample utility being the 'Trader' service which makes possible the location of objects exhibiting certain required properties — other CORBA services defined are event, naming, life cycle, persistence, time, relationship, property, query, security, collections, licensing, transaction, concurrency and externalisation,
- common facilities, similar to objects services, but more useful for end-user applications,
- application objects, which are objects developed as part of the end-user applications.

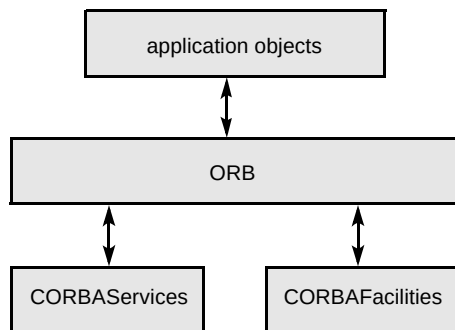


Fig 4 The CORBA infrastructure model.

The subtle difference between CORBA and DCE is, whereas DCE implementations are based on a standard reference implementation provided by the Open Group, CORBA just defines the architecture and interfaces. ORB vendors are free to define how they implement their products as long as compliance to the CORBA standard specification is met.

Figure 5 shows a request being sent from a client to an object implementation. The object implementation is, in practical terms, the executable server code together with the data necessary to perform the desired actions requested by the client. The ORB is responsible for all of the mechanisms required to find the object implementation needed to satisfy the request, and to communicate the data making up the request. The client interface is completely independent of the location of the target object or the language in which it is implemented.

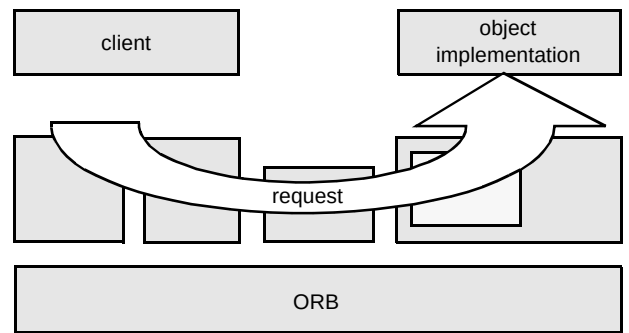


Fig 5 A client makes a request through an ORB.

Figure 6 shows a typical structure of the interfaces of an ORB. These are:

- dynamic invocation interface (DII) which allows the client call to specify the object to be invoked, the operation to be performed and the set of parameters required in a dynamic manner, rather than calling a stub routine specific to a particular operation on a particular object — this allows the clients to discover and invoke methods on servers at run-time without using generated stub code,
- client IDL stubs are used to describe how clients invoke services on servers, with each interface used by the client being described using IDL — the stubs also provide for any necessary marshalling and un-marshalling,
- the ORB interface provides the interfaces to local services, e.g. the naming service allows objects to be identified by name rather than by object reference,
- IDL skeletons are similar to client stubs, providing for static invocation, and are generated by the IDL compiler,
- object adapters pass requests to the servers and provide the primary way for object implementations to access the services provided by the ORB.

Not shown in Fig 6 are:

- the interface repository, which allows manipulation of registered interfaces,
- the implementation repository, which provides run-time information about the classes a server supports and the objects that are instantiated,
- the dynamic skeleton interface (DSI), which can be viewed as the server equivalent to the DII, and which is used to satisfy client requests to services that have no server IDL skeleton stubs.

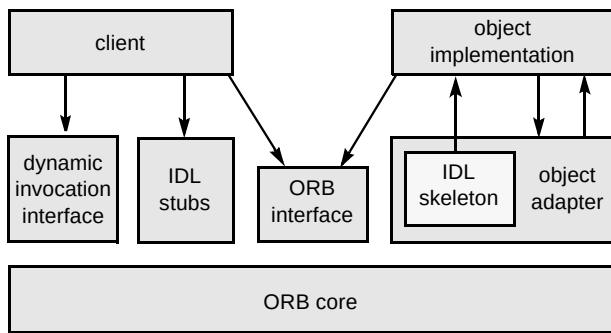


Fig 6 ORB interfaces.

In CORBA, the interaction between a client and an object is achieved in a similar way to that used in RPC-based technologies such as DCE, namely the interface. DCE uses the interface to define the function and its parameters, but CORBA uses the interface to define the object, the methods/operations that can be invoked on the object, and the parameters that are input, output, or both input and output. The interface that the client sees is completely independent of where the object is located, in what programming language it is defined, and how its services are implemented. Like DCE, CORBA describes interfaces using an interface definition language. CORBA IDL is a subset of ANSI C++, with additional constructs to support the operation/invoke mechanism. It also provides support for standard C++ preprocessing features. There are CORBA IDL language mappings to C++, C and Smalltalk. A simple IDL specification might be:

```

// IDL
interface Customer{
    // Attributes to hold a customer's name and balance of
    // credit for a telephone //
    debit card service,
    attribute long customerCredit;
    readonly attribute string customer;

    // The operations defined on the interface,
    void increaseCredit (in float amount,
                        out float newCredit);
    void decreaseCredit (in float amount,
                        out float newCredit);
};
  
```

Of particular importance from the developer's point of view is that not only are application objects invoked using interfaces and IDL, but the services of CORBA are also invoked using IDL. There is thus one common paradigm used throughout CORBA, i.e. object invocation using the CORBA IDL.

Client/server communication using an ORB can be achieved in a number of ways. If the object to be invoked is known at compile time (static method), then the object's name can be passed as a parameter to a 'bind' call, which

will return an object reference. However, if the object's name is not known, its object reference may be obtained by using the 'Trader' service.

Using the static method, the object reference would be obtained in the following manner:

```
ObjRef = _bind ('ServiceIamCalling', Hostname);
```

The call to the object adapter would provoke instantiation of an instance of the required object, by locating details from the implementation repository. The instance is then set running and a reference to the instance returned. When the client makes a request, the local proxy implemented by the stub code handles the call. The proxy acts as the remote server object. The stubs provide any marshalling or unmarshalling and the request is passed to the ORB, which then passes request information via the object adapter and IDL skeleton to the object implementation. The method is then run and returns the results.

The dynamic invocation method is slightly different, requiring construction and population of a request object with the necessary interface, methods, and parameters of the object, before an invoke call can be made.

Interoperability

In a true heterogeneous distributed system based on CORBA ORB technologies a basic requirement would be that ORBs purchased from different vendors, supported upon different platforms, should be able to interoperate. How this is achieved in CORBA 2.0 is defined by the ORB interoperability architecture, which also supports environment-specific inter-ORB protocols (ESIOPs) that are optimised for particular environments such as the DCE, support for inter-ORB bridges and general inter-ORB protocols (GIOPs) and Internet inter-ORB protocols (IIOPs). The GIOP may be mapped on to different transports, such as IP, specifying the appropriate protocol elements. However, GIOP on its own does not provide for interoperability. IIOP is a realisation of the abstract definitions provided by GIOP. Figure 7 illustrates the relationships between GIOP and IIOP.

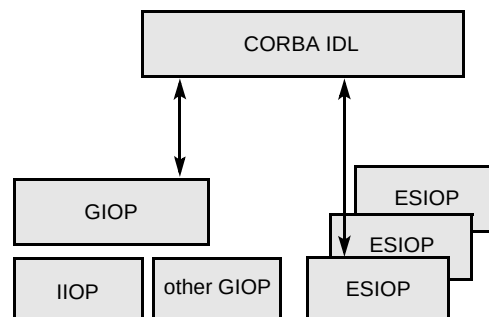


Fig 7 Inter-ORB protocol relationships.

The IIOP model is based upon two main message types — a request from a client and a reply from the server — together with additional support to handle exceptions.

To begin to establish a connection, the client issues a request for a named object from the naming services. The naming service looks up the name in its directory and, if the name is 'remote', uses an IIOP engine that is provided by the ORB to find the object in other naming services directories. The remote naming service returns the interoperable object reference, which effectively, gives the address of the object on the network (host name, port and object name) to the client via the IIOP engine. The client can then communicate directly with the remote object using the interoperable object reference.

CORBA 3.0

Ten years after the birth of the OMG, the CORBA 3.0 architecture is due to be released in 1999, promising to be a mature version of the CORBA standard, underpinned with enough architectural technology to facilitate development of 'enterprise-level', production-quality middleware. A number of important technology gaps appear to have been addressed, notably:

- support for building Internet applications where IIOP is able to securely traverse corporate firewalls — this is based on the IONA WonderWall TM product,
- many organisations use CORBA products with messaging products such as OrbixTalk and IBM's MQSeries — this addition to the specification is the first meaningful industry standard for message-oriented middleware (MOM) and defines the use of messaging technology within the CORBA framework,
- CORBA 3.0 specifies a component architecture that is a superset of Enterprise Java Beans (EJB), CORBABeans, which will mean that CORBA can enable developers to build applications made out of a wide variety of different components regardless of the language in which they were developed or on what platform — the OMG is specifying a component model and container that can be mapped on to various programming languages (the Java mapping of the CORBA component model will be EJB compliant, by way of a thin layer on top of the existing CORBA model, which has always been a fundamentally component-oriented architecture since its creation in 1991),
- a CORBA-based ORB has been adopted by many organisations as the foundation of cross-platform integration strategies, addressing the integration of legacy systems and the developing enterprise —

CORBA also provides organisations with interoperability with other organisations and future support for electronic commerce applications.

CORBA undoubtedly provides powerful support for distributed applications. However, the first generation of CORBA implementations can be slow and inefficient, being generally unsuitable for mission-critical client/server environments. Their adoption for use in 'real-time' or even 'soft real-time' telecommunications applications has been prejudiced by their indifferent performance. Additionally, CORBA ORBs do not generally support such things as fail-over, automatic restart of servers, load balancing, buffer optimisation, automatic memory management, and support is only provided for TCP/IP.

3.3 COM

COM (Component Object Model) is both a specification and implementation developed by Microsoft Corporation. COM provides a framework for integrating components and provides support for interoperability and reusability of distributed objects. COM allows developers to build systems by assembling reusable components from different vendors. COM defines an application programming interface (API) allowing creation of components for use in integrating applications or to allow diverse components to interact. In order to interact, components must conform to a binary structure specified by Microsoft. The binary compatibility specification and associated implementation allows clients to invoke services provided by COM-compliant components (COM objects). As shown in Fig 8, services implemented by COM objects are exposed through a set of interfaces that represent the only point of contact between clients and the target object.

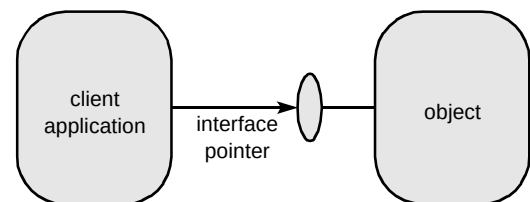


Fig 8 Client calls a COM object through an interface pointer.

COM defines a binary structure for the interface between the client and the object and it is this binary structure that provides the basis for interoperability between software components written in arbitrary languages. As long as a compiler can reduce language structures down to this binary representation, the implementation language for clients and COM objects does not matter, the point of contact being the run-time binary representation. COM objects and clients can then be coded in any language that supports Microsoft's COM binary structure. COM objects can support any number of interfaces, where an interface provides a grouped collection of related methods.

The Component Object Model is one of the most popular commercial object architectures, but many questions remain unanswered about its reliability, cross-platform support, and even scalability. Microsoft hope that some of these doubts have been addressed by COM+ which extends and enhances existing services, including:

- a 'publish and subscribe' service, which provides a general event mechanism that allows multiple clients to 'subscribe' to various 'published' events — when the publisher fires an event, the COM+ events system iterates through the subscription database and notifies all subscribers,
- an in-memory database (IMDB), with support for transactions, which provides an application with fast access to data, without incurring the overhead associated with storing and accessing durable state to and from physical disk,
- queued components, which allow clients to invoke methods on COM components using an asynchronous model — such a model is particularly useful on unreliable networks and in disconnected usage scenarios,
- dynamic load balancing, which automatically spreads client requests across multiple equivalent COM components,
- full integration of MTS into COM, which includes broader support for attribute-based programming, improvements in existing services such as transactions, security and administration, as well as improved interoperability with other transaction environments through support for the transaction Internet protocol (TIP).

3.4 DCOM

Microsoft's architecture for distributed component-based communication is DCOM (Distributed Component

Object Model) and is based on COM. DCOM is not a separate product, but is an implementation of Microsoft's distributed architecture, extending COM to allow network-based component interaction. COM processes can run on the same machine but in different address spaces, while the DCOM extension allows processes to be spread across a network, with components operating on a variety of interacting platforms, as long as DCOM is available within the environment. Figure 9 shows this feature.

COM and DCOM are best considered as a single technology providing a range of services for component interaction, from services promoting component integration on a single platform, to component interaction across heterogeneous networks. COM and the DCOM extensions are merged into a single run time providing both local and remote access.

DCOM is reasonably new technology, and as such may need to evolve before it can be described as enterprise-level or infrastructure middleware. Many of the services and much of the support required are currently not provided, e.g. end-to-end security, load balancing, and advanced fault-handling services such as fail-over and automatic restart and recovery. Many applications are likely to be built using DCOM because of its Microsoft roots and potential desktop market. However, the very proprietary nature of the complete product set will make it less attractive to some.

3.5 Java RMI

Java Remote Method Invocation (RMI) is middleware for Java, enabling developers to build small-scale distributed applications [7].

RMI applications often follow the client/server model. Server applications create remote Java objects, and make the references to them accessible. The server then waits for clients to invoke methods on these remote Java objects. Client applications obtain a remote reference to one or more

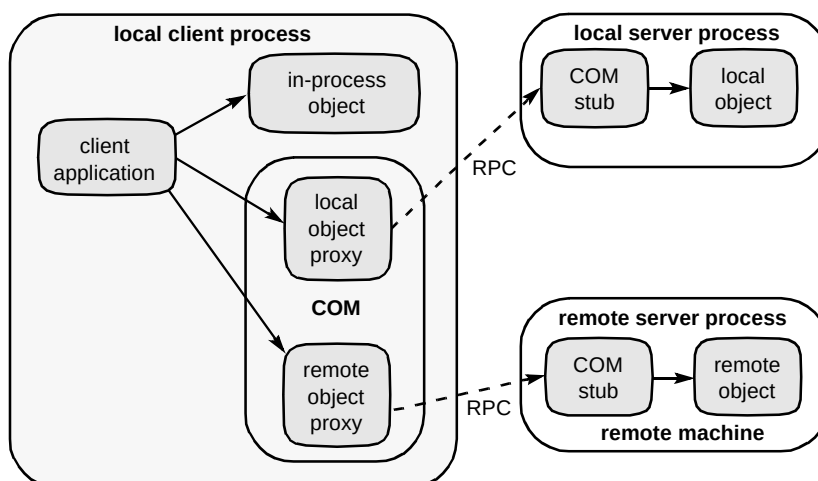


Fig 9 COM extended by DCOM.

remote Java objects in the server and then invoke methods on them using input and output parameters. Synchronous processing is supported by RMI, in a manner similar to an RPC mechanism. RMI supports much of the functionality found in an ORB, providing the basis of an ORB for Java applications but without the need for factories. RMI clients interact with RMI servers by calling the methods of local copies of remote objects — these are termed proxies.

Figure 10 shows an RMI distributed object application that uses the registry to obtain a reference to a remote Java object. The server calls the registry to associate (or bind) a name with a remote object. The client looks up the remote object by its name in the server's registry and then invokes a method on it.

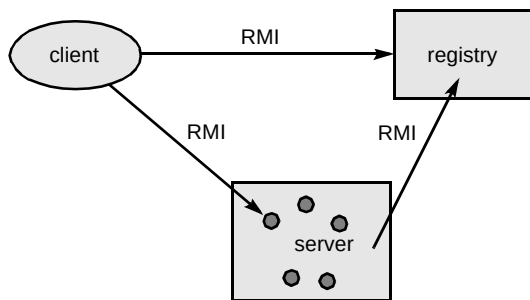


Fig 10 RMI client locating a remote object and invoking a method upon it.

RMI is a form of ORB, but for one programming language and one environment, namely the Java System. RMI has limitations as middleware in that some of the expected run-time services, such as load balancing, fail-over and automatic multithreading, are not present, although the product is evolving and has the potential to rival technologies such as CORBA and DCOM.

4. Current use of 'middleware' in telecommunications

The Internet may be considered as yet another network, adding its own flavour of services to the overall set of communications services. However, before essential business services can be introduced on to this medium, significant improvements in the areas of security, resource management and charging need to be made. Ericsson's 'IP@Service' platform provides a framework for developing and deploying commercial IP-based services, adding support for charging and subscription services. The platform is a topical implementation of distributed systems technology, using CORBA 2.0 to implement a distributed object environment and IIOP to achieve object interoperability (see Fig 11).

Most applications and platform components are implemented using Java to achieve maximum operating system independence and portability. An example of the use of distributed systems technology within the platform is the 'brokering' function, giving end users access to the services

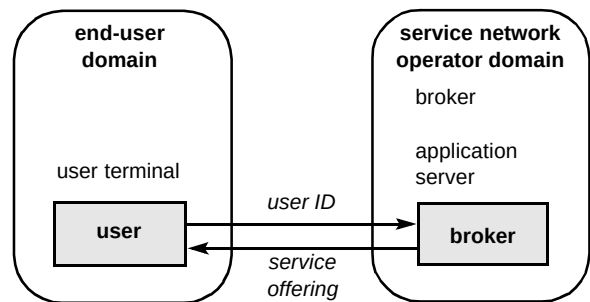


Fig 11 The 'IP@Service' broker function.

provided by service providers in the network. High availability of the broker service is achieved by use of an enhanced CORBA-naming service.

Brokers have a client side and a server side. The client side handles interface to smart cards, which are used for identifying end users and for presenting the applications available to them. The server side handles authentication and controls access to the list of available applications. End-user profiles, which include the list of application subscriptions, are retrieved from the user database.

By using a smart card, end users can invoke a service network access application that uses a security API for signing messages sent to the broker. Using an X.500 database, the broker verifies incoming log-on messages with the security API. The broker then retrieves the user profile from the database. Based on user and terminal profiles, the broker performs a filtering function to derive the most suitable service offering. The service offering is returned to the user, who may then select and invoke a service from the list.

5. Conclusions

Middleware is there to help solve many application connectivity and interoperability problems. However, there is a gap between promise and practice. Many popular middleware products use proprietary implementations, which make applications dependent on a single vendor's product and forces dependence on the vendor for maintenance support and future enhancements. This reliance could have a negative effect on a system's flexibility and maintainability. Middleware products raise the level of abstraction of programming distributed applications, but they still leave the application developer with hard design choices in terms of which middleware services to use to meet their required functionality, out of the broad range on offer. The costs of using middleware technology in terms of licence fees, system development and ongoing maintenance are dependent on the required operating systems, the types of platform and complexity of the application, and can be limiting. However, when evaluated against the cost of developing an in-house middleware solution the potentially negative effect may be viewed as acceptable.

Middleware services are not the panacea that marketing hype has led us to believe, but as they evolve and mature they promise an increasingly rich and flexible way to provide highly functional, performant and innovative services.

Acknowledgements

The author would like to thank Alex Cruse and Richard Swale for their help and advice in composing this material.

References

- 1 Crowcroft J: 'Open Distributed Systems', UCL (1996).
- 2 Ovum reports — <http://www.ovum.com>
- 3 Lockhart H W: 'OSF DCE, Guide to Developing Distributed Applications', McGraw-Hill (1994).
- 4 Rosenberry W and Teague J: 'Distributing applications across DCE and WindowsNT', O'Reilly (1993).
- 5 OMG — <http://www.omg.org>
- 6 OMG: 'The Common Object Request Broker: Architecture and Specification', Revision 2.0 (1994).
- 7 Java RMI — <http://www.javasoft.com>



In 1989 Steve Condie moved to BT Laboratories (BTL) from London where he had managed a CSS mainframe migration project.

His work at BTL specialised in interworking between legacy mainframe and newer, Unix-based application systems. His work also involved investigation of network management issues. He later joined the Distributed Processing Technology group, investigating evolving distributed technologies and specialised in distributed on-line transaction processing (OLTP) systems.

In 1994 he joined the NIEC, investigating the application of distributed systems technology to Network Intelligence platforms. In 1997 the focus of his work changed to addressing the challenges of voice-over-IP technologies and evolving standards. He currently leads work investigating the use of PSTN/IP gateways and also represents BT in the ETSI Tiphon IP Telephony standardisation project.